Group project - Wine - Jared Martin - R Code

Jared Martin

2024-11-24

```
library(tidyverse)
```

```
## -- Attaching core tidyverse packages ------------------------ tidyverse 2.0.0 --
## v dplyr     1.1.4     v readr     2.1.5
## v forcats   1.0.0     v stringr   1.5.1
## v ggplot2   3.5.1     v tibble    3.2.1
## v lubridate 1.9.3     v tidyr     1.3.1
## v purrr     1.0.2
## -- Conflicts ------------------------------------------ tidyverse_conflicts() --
## x dplyr::filter() masks stats::filter()
## x dplyr::lag()    masks stats::lag()
## i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become errors
```

```
library(class)
library(tree)
```

Cleaning and Preparing the data

The following code pulls in the two wine data sets, creates a new variable called wine type (red or white), makes quality and type factor variables, joins the two data sets together and saves the data as a csv.

```
rbind(read.csv("./Data/winequality-red.csv", sep = ";") %>%
                     mutate(wine.type = "red"),
                   read.csv("./Data/winequality-red.csv", sep = ";") %>%
                     mutate(wine.type = "white"))  %>%
  write.csv( "./Data/wine_data.csv")

wine_data <- read_csv("./Data/wine_data.csv")
```

```
## New names:
## Rows: 3198 Columns: 14
## -- Column specification
## -------------------------------------------------------- Delimiter: "," chr
## (1): wine.type dbl (13): ...1, fixed.acidity, volatile.acidity, citric.acid,
## residual.sugar...
## i Use `spec()` to retrieve the full column specification for this data. i
## Specify the column types or set `show_col_types = FALSE` to quiet this message.
## * `` -> `...1`
```

```r
wine_data$quality <- factor(wine_data$quality)

wine_data$wine.type <- factor(wine_data$wine.type)

wine_data[,2:14] -> wine_data

wine_data %>%
  dplyr::select(quality, everything()) -> wine_data
```

Research Question #1: Can we predict the quality of a wine given its characteristic?

Each wine was given as ranking for quality between 0-10. Although in the data set the lowest level of quality is 3, while the highest is an 8. Given that there is more then two level of quality we will be using KKN and classification trees to predict the quality of the wine given it's characteristics.

```r
levels(wine_data$quality)
```

```
## [1] "3" "4" "5" "6" "7" "8"
```

Method #1: KNN

1. Spliting the data into a testing and training dataset.

```r
set.seed(1)

n <- nrow(wine_data)

z <- sample(n, n/2)

wine_test <- wine_data[z,]

wine_train <- wine_data[-z,]

x.train <- wine_train[,2:12]
x.test <- wine_test[,2:12]

y.train <- wine_train$quality
y.test <- wine_test$quality

dim(x.train)[1] == length(y.train)  # Test to see if length is the same. Should be TRUE
```

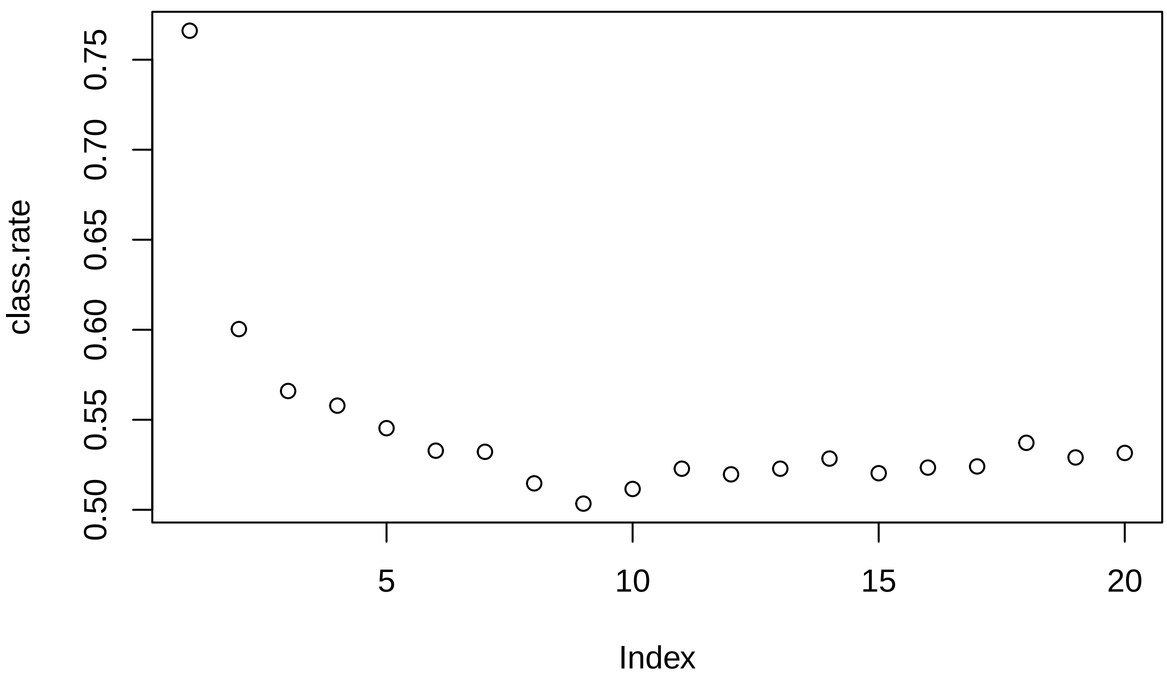
```
## [1] TRUE
```

```r
dim(x.test)[1] == length(y.test)    # Test to see if length is the same. Should be TRUE
```

```
## [1] TRUE
```

2. Determining the optimal number of nearest neighbors "K"

```
class.rate = rep(0,20)

for (K in 1:20) {
  knn.result = knn(x.train, x.test, y.train, K)
  class.rate[K] = mean( y.test == knn.result )
}

plot(class.rate)
```




```
max(class.rate) # optimal number of K neighbors is 1
```

```
## [1] 0.7661038
```

4. Cross Validation - CClasification Rate and Confusion table using Optimal number of nearest Neighbors.

```
knn_cv_wine_quality <- knn(x.train, x.test, y.train, 1)

table(y.test, knn_cv_wine_quality)
```

```
##       knn_cv_wine_quality
## y.test   3   4   5   6   7   8
##      3   7   4   2   0   0   0
##      4   0  30  12  10   2   0
##      5   2   8 541  90  16   0
##      6   2  12 116 501  18   6
```

```
##      7    0    2   20   40  138    0
##      8    0    0    0    6    6    8
```

```
knn_cv_classification_rate <- mean(y.test == knn_cv_wine_quality)

# Classification rate of .766
```

3. LOOCV - CClasification Rate and Confusion table using Optimal number of nearest Neighbors.

```
y.test <- wine_data$quality

x.train <- wine_data[,2:12]

knn_loocv_wine_quality <- knn.cv(x.train, y.test, k = 1, prob = FALSE)

table(y.test, knn_loocv_wine_quality)
```

```
##       knn_loocv_wine_quality
## y.test    3    4    5    6    7    8
##      3   20    0    0    0    0    0
##      4    0  106    0    0    0    0
##      5    0    0 1362    0    0    0
##      6    0    0    0 1276    0    0
##      7    0    0    0    0  398    0
##      8    0    0    0    0    0   36
```

```
knn_cv_classification_rate <- mean(y.test == knn_loocv_wine_quality)

#Wow! Classification rate of 1!
```

Method #2: Classification Trees

1. Classification Tree for Quality

```
tree_wine_quality <- tree(quality ~ ., data = wine_train)

tree_wine_quality
```

```
## node), split, n, deviance, yval, (yprob)
##       * denotes terminal node
##
##  1) root 1599 3736.00 5 ( 0.004378 0.032520 0.440901 0.388368 0.123827 0.010006 )
##    2) alcohol < 10.525 996 1887.00 5 ( 0.006024 0.032129 0.601406 0.322289 0.035141 0.003012 )
##      4) sulphates < 0.615 566  898.10 5 ( 0.005300 0.044170 0.687279 0.259717 0.003534 0.000000 )
##        8) total.sulfur.dioxide < 98.5 479  818.40 5 ( 0.006263 0.052192 0.640919 0.296451 0.004175 0
##         16) sulphates < 0.525 178  278.10 5 ( 0.005618 0.095506 0.747191 0.146067 0.005618 0.000000 )
##         17) sulphates > 0.525 301  501.50 5 ( 0.006645 0.026578 0.578073 0.385382 0.003322 0.000000 )
```

```
##         9) total.sulfur.dioxide > 98.5 87   38.27 5 ( 0.000000 0.000000 0.942529 0.057471 0.000000 0.0
##       5) sulphates > 0.615 430  902.50 5 ( 0.006977 0.016279 0.488372 0.404651 0.076744 0.006977 )
##        10) alcohol < 9.85 243  419.40 5 ( 0.004115 0.016461 0.625514 0.325103 0.024691 0.004115 )
##          20) fixed.acidity < 10.75 220  340.60 5 ( 0.004545 0.018182 0.681818 0.281818 0.013636 0.0000
##          21) fixed.acidity > 10.75 23   38.54 6 ( 0.000000 0.000000 0.086957 0.739130 0.130435 0.0434
##        11) alcohol > 9.85 187  430.10 6 ( 0.010695 0.016043 0.310160 0.508021 0.144385 0.010695 ) *
##     3) alcohol > 10.525 603 1463.00 6 ( 0.001658 0.033167 0.175788 0.497512 0.270315 0.021559 )
##       6) volatile.acidity < 0.385 222  488.90 7 ( 0.000000 0.009009 0.090090 0.396396 0.468468 0.0360
##       7) volatile.acidity > 0.385 381  889.80 6 ( 0.002625 0.047244 0.225722 0.556430 0.154856 0.0131
##        14) alcohol < 11.45 208  428.60 6 ( 0.004808 0.062500 0.298077 0.581731 0.052885 0.000000 ) *
##        15) alcohol > 11.45 173  405.70 6 ( 0.000000 0.028902 0.138728 0.526012 0.277457 0.028902 )
##          30) sulphates < 0.635 86  183.10 6 ( 0.000000 0.058140 0.232558 0.593023 0.116279 0.000000 )
##          31) sulphates > 0.635 87  178.30 6 ( 0.000000 0.000000 0.045977 0.459770 0.436782 0.057471 )
```

```r
summary(tree_wine_quality)
```

```
##
## Classification tree:
## tree(formula = quality ~ ., data = wine_train)
## Variables actually used in tree construction:
## [1] "alcohol"              "sulphates"            "total.sulfur.dioxide"
## [4] "fixed.acidity"        "volatile.acidity"
## Number of terminal nodes:  10
## Residual mean deviance:  1.829 = 2906 / 1589
## Misclassification error rate: 0.3952 = 632 / 1599
```

```r
# not all class are represented as terminal node classifications (3, 4 & 8 are not represented).
```

2. Calculating the Classification Rate for Unprunned Tree

```r
tree_hat <- predict(tree_wine_quality, wine_test, type = "class")

table(tree_hat, wine_test$quality)
```

```
##
## tree_hat   3   4   5   6   7   8
##        3   0   0   0   0   0   0
##        4   0   0   0   0   0   0
##        5   6  35 483 233   9   0
##        6   7  19 148 322 103  12
##        7   0   0  26 100  88   8
##        8   0   0   0   0   0   0
```

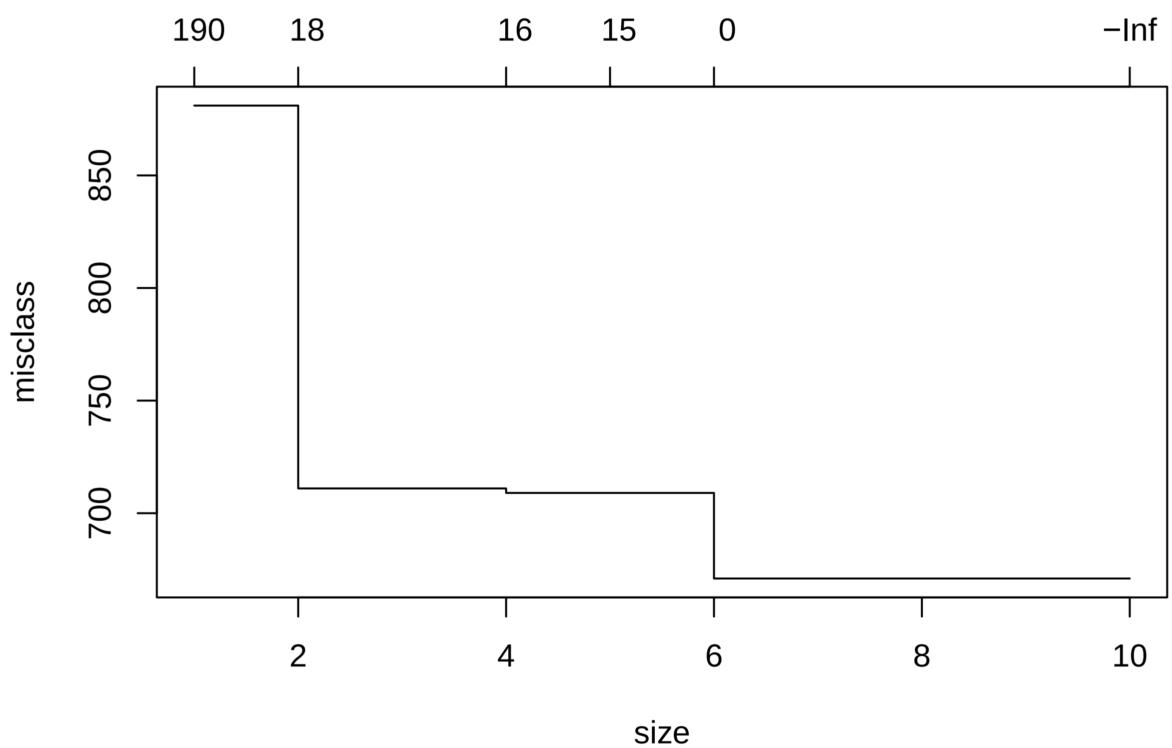
```r
tree_classification_rate <- mean(tree_hat == wine_test$quality)
```

3. Prunning the Classification tree on misclassifaction rate.

```r
# The optimal number of terminal nods on a tree is 8. The same number of terminal nods as the tree abov

set.seed(1)

tree_cv_wine_quality <- cv.tree( tree_wine_quality, FUN = prune.misclass )

tree_cv_wine_quality
```

```
## $size
## [1] 10  6  5  4  2  1
##
## $dev
## [1] 671 671 709 709 711 881
##
## $k
## [1]  -Inf   0.0  15.0  16.0  18.5 194.0
##
## $method
## [1] "misclass"
##
## attr(,"class")
## [1] "prune"         "tree.sequence"
```

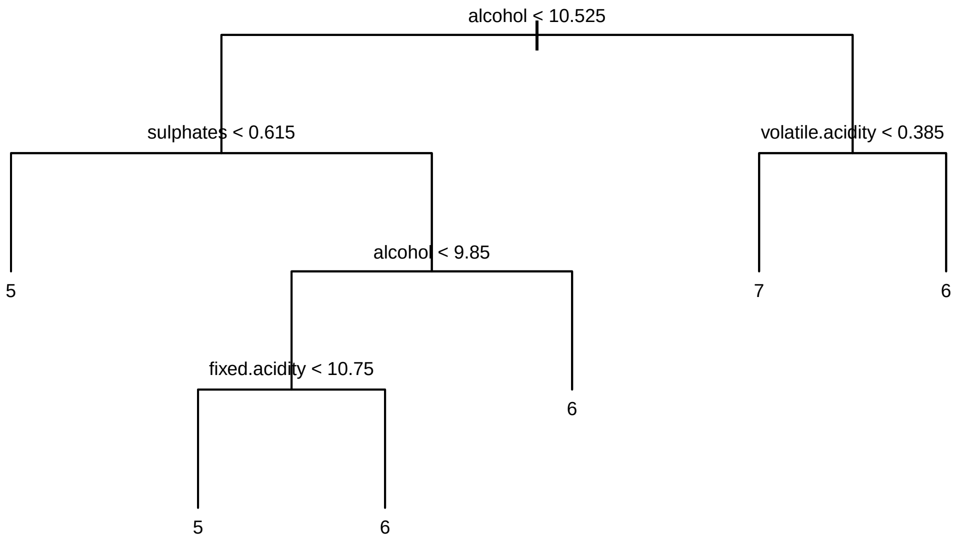
```r
plot(tree_cv_wine_quality)
```




```r
Tree_pruned_wine_quality <- prune.misclass(tree_wine_quality, best = 6)
```

4. Plotting Classification Tree

```r
plot(Tree_pruned_wine_quality, type = "uniform")
text(Tree_pruned_wine_quality, cex = 0.6)
```



5. Cross Validation of the Classification Tree

```r
tree_hat <- predict(Tree_pruned_wine_quality, wine_test, type = "class")

table(tree_hat, wine_test$quality)
```

```
##
## tree_hat   3   4   5   6   7   8
##        3   0   0   0   0   0   0
##        4   0   0   0   0   0   0
##        5   6  35 483 233   9   0
##        6   7  19 148 322 103  12
##        7   0   0  26 100  88   8
##        8   0   0   0   0   0   0
```

```r
tree_pruned_classification_rate <- mean(tree_hat == wine_test$quality)
```